

ISLAMIC UNIVERSITY OF TECHNOLOGY



SWE 4301

LECTURE 17

Topic: Hierarchical Smells

March 15, 2025

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1 Hierarchical Smells	3
1.1 Missing Hierarchy	4
1.1.1 Rationale	4
1.1.2 Potential Causes	4
1.1.3 Example	4
1.2 Unnecessary Hierarchy	5
1.2.1 Rationale	5
1.2.2 Potential Causes	6
1.2.3 Example	6
1.3 Unfactored Hierarchy	6
1.3.1 POTENTIAL CAUSES	6
1.3.2 Example	7
1.4 Wide Hierarchy	7
1.4.1 Example	8
1.5 Speculative Hierarchy	8
1.5.1 Potential Causes	8
1.5.2 Example	9
1.6 Deep Hierarchy	9
1.6.1 Potential Causes	10
1.6.2 Example	11
1.7 Rebellious Hierarchy	11
1.7.1 Potential Causes	11
1.7.2 Example	12
1.8 Broken Hierarchy	12
1.8.1 Potential Causes	13
1.8.2 Example	13
1.9 Multipath Hierarchy	13
1.9.1 Potential Causes	14
1.9.2 Example	14

1 HIERARCHICAL SMELLS

A hierarchical organization offers a powerful tool for dealing with complex diversity in real-world entities. It helps order the entities, deal with similarities and differences, and think in terms of generalizations and specializations.

Modern software is extremely complex and requires a way to understand, reason about, and analyze the relationships between a large number of real-world entities. Organizing these entities in a hierarchy is one of the key tools that help deal with this complexity during software development.

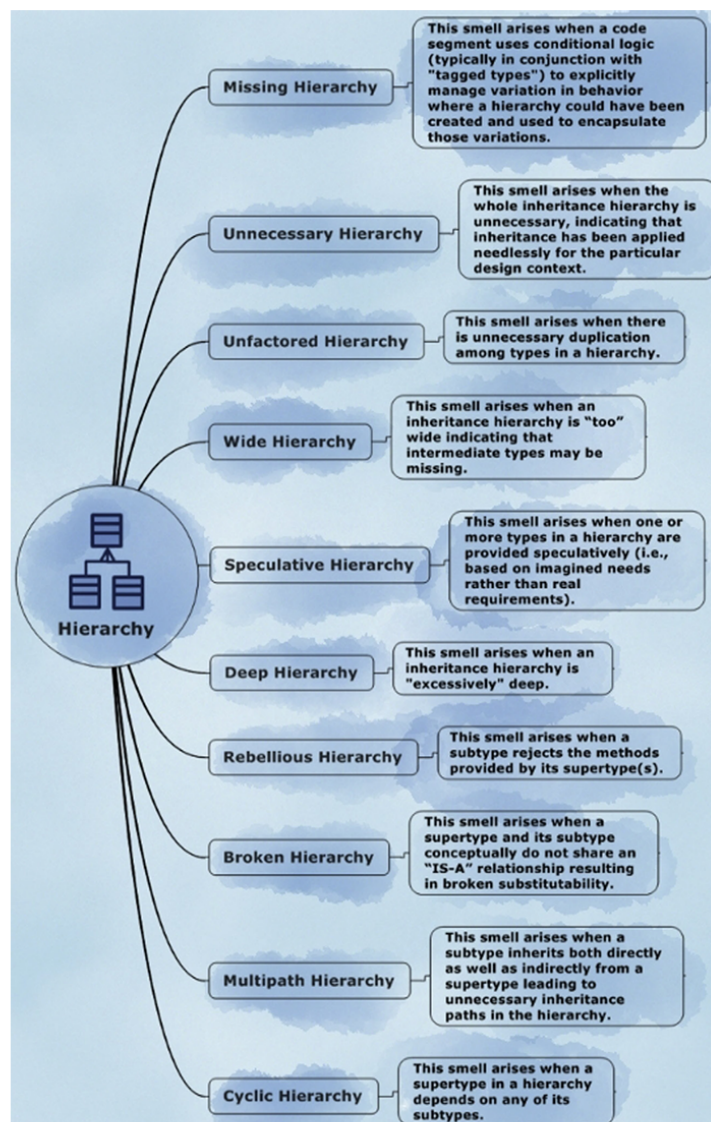


FIGURE 6.3

Smells resulting due to the violation of principle of hierarchy.

1.1 Missing Hierarchy

This smell arises when a code segment uses conditional logic (typically in conjunction with “tagged types”) to explicitly manage variation in behavior where a hierarchy could have been created and used to encapsulate those variations.

1.1.1 Rationale

Switch-based-on-type-codes (or chained if-else statements) is one of the most well-known design smells. When type information is encoded (for example, using an enumeration, an integral value, or a string), variation in behavior is not encapsulated properly.

Using “tagged types” or conditional logic to handle variation in behavior clearly indicates that the enabling technique “apply meaningful classification” was not employed and as a result a hierarchy is “missing” in the design. Hence, we name this smell Missing Hierarchy.

1.1.2 Potential Causes

- **Misguided simplistic design**

Inexperienced designers often adopt encoded type values that are used in conjunction with conditional statements and (wrongly) believe that it results in a “straight forward” or “simple” design.

- **Procedural approach to design**

In C-like procedural languages, the main way to handle variation in behavior is to use encoded types and conditional logic. Following such a design approach while programming in object-oriented languages is a common cause of this smell.

- **Overlooking inheritance as a design technique**

We have come across many cases where developers were unaware of the benefits inheritance can bring to design when used correctly. Due to this, developers often rely on explicit conditional logic for handling variation instead of creating and using a hierarchy.

1.1.3 Example

```
1 class Circle{  
2     public void draw() {  
3         System.out.println("Drawing a Circle");  
    }
```

```
4     }
5 }
6 class Square{
7     @Override
8     public void draw() {
9         System.out.println("Drawing a Square");
10    }
11 }
12 class DrawShape {
13     String shapetype;
14     DrawShape(String shapetype){
15 this.shapetype=shapetype;
16    }
17     public void draw() {
18 if(shapetype.equals('circle')) new Circle().draw();
19 else if(shapetype.equals('square')) new Square().draw();
20    }
21 }
22 //Rewriting DrawShape class
23 class DrawShape {
24     Shape shape;
25     DrawShape(Shape shape){
26 this.shape=shape;
27    }
28     public void draw() {
29 shape.draw();
30    }
31 }
```

1.2 Unnecessary Hierarchy

This smell arises when the whole inheritance hierarchy is unnecessary, indicating that inheritance has been applied needlessly for the particular design context.

1.2.1 Rationale

For effective application of the principle of hierarchy, it is important to adhere to the enabling technique “apply meaningful classification.” For a “meaningful” application of classification, the focus should be more on capturing the commonalities and variation in behavior rather than data. When this enabling technique is violated, it can result in an inheritance hierarchy that is “unnecessary” and which needlessly complicates the design.

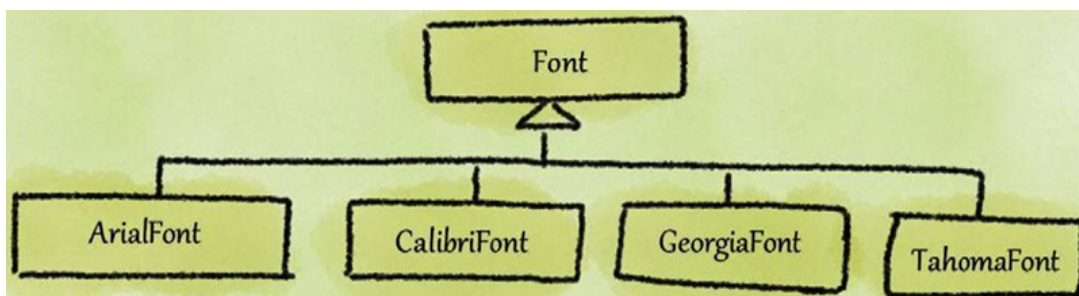
1.2.2 Potential Causes

Subclassing instead of instantiating

Let us take a simple example of representing roses with different color such as red, pink, and white. Assume that these different kinds of roses differ only in their color and not in their behavior. Hence, it will be inappropriate to create a Rose supertype with subtypes such as RedRose, PinkRose, and WhiteRose. Instead of creating sub classes, a better design would be to create objects of type Rose with an attribute named color that can take values such as red, pink, and white.

1.2.3 Example

Font is the supertype and fonts such as Arial and Calibri are supported via Font's subtypes named ArialFont and CalibriFont. However, if we were to reflect on this design, we would realize that since the editor does not change its behavior based on the font, classifying the types of fonts within a hierarchy is unnecessary.



1.3 Unfactored Hierarchy

This smell arises when there is unnecessary duplication among types in a hierarchy. There are two forms of this smell:

- Duplication in sibling types: Sibling types in the hierarchy have similar code segments that can be moved into one of their supertypes.
- Duplication in super and subtypes: There are similar code segments in super and subtypes indicating redundant code in the subtypes.

1.3.1 POTENTIAL CAUSES

- Copying subtypes

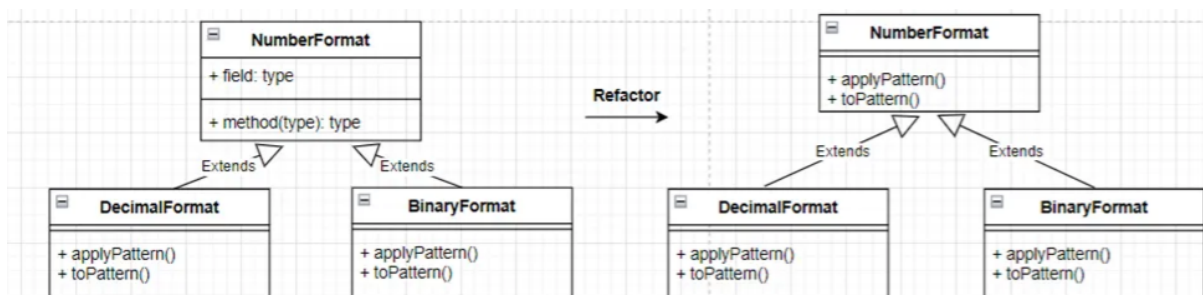
We have seen developers copy the code of a subtype and paste it as a renamed new subtype with necessary changes. Of course, this would “get the work done,” but this duplication will introduce this smell. This often occurs when excessive focus is placed on just completing the work without due attention to design quality.

- **Improper handling of outliers** Consider an example wherein a superclass named Bird has derived classes such as Pigeon, Crow, and Sparrow. The superclass Bird has a method fly(). Now, if a new subclass Penguin (which is a non-flying bird) needs to be supported in the design, then we need to force-fit the Penguin class in the hierarchy by providing an empty body for fly() or rejecting the method, such as by throwing an exception.

An alternative solution for this is to remove the fly() method in the Bird class and instead define fly() method in all subclasses except Penguin. However, this leads to unnecessary duplication of fly() method in the sibling classes in the hierarchy.

We could introduce intermediate classes FlyingBird and NonFlyingBird as subclasses of Bird class, and have only the FlyingBird class support the fly() method.

1.3.2 Example



1.4 Wide Hierarchy

This smell arises when an inheritance hierarchy is “too” wide, indicating that intermediate types may be missing.

When generalization is inadequately applied and consequently intermediate types are missing, it can result in the following problems:

- Missing intermediate types may force the clients of that hierarchy to directly refer to the subtypes. This dependency affects the changeability and extensibility of the hierarchy.

- There may be unnecessary duplication within types (since commonality cannot be properly abstracted due to a lack of intermediate types).

For effective design of inheritance hierarchies, we need to “maintain moderation in size.” If there are a large number of subtypes at the same level, it becomes harder to understand and use the hierarchy. Since the hierarchy is “too wide,” this smell is named Wide Hierarchy. Based on our experience, we consider a hierarchy wide if any type in the hierarchy has more than 9 immediate subtypes.

Both Wide Hierarchy and Unfactored Hierarchy are similar in one aspect: both violate the enabling technique “apply meaningful generalization.” However, the difference is that wide hierarchy arises when there are “too” many immediate subtypes for a supertype (indicating that intermediate types may be missing), whereas Unfactored Hierarchy arises when there is duplication in the hierarchy. In fact, both of them often occur together.

1.4.1 Example

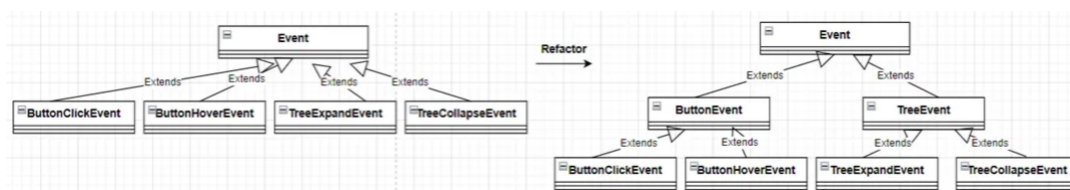


Figure 1: Wide Hierarchy diagram

1.5 Speculative Hierarchy

This smell arises when one or more types in a hierarchy are provided speculatively (i.e., based on imagined needs rather than real requirements).

One of the key enabling techniques for the effective application of the principle of hierarchy is “apply meaningful generalization.” Generalization should be performed for exploiting commonalities in existing types. Further, in addition to current needs, generalization should also take into account the planned future needs of the product. However, when this is not followed and generalization is applied purely based on imagined requirements, the resulting hierarchy may have supertypes that aren’t needed at all.

1.5.1 Potential Causes

- **Future-proofing**

This design smell often arises due to attempts to make the design of the system “future-proof,” i.e., a design that can accommodate all changes one could imagine will happen in the future.

- **Over-engineering**

When designers over-engineer a design and provide generalized types that aren’t actually needed to address the real requirements, it can lead to a Speculative Hierarchy smell.

1.5.2 Example

A Chart interface is implemented by LineChart and BarChart. Currently BarChart is not even required in project, but developer created this hierarchy keeping future requirements in mind or based on imagined needs. To refactor it, just removed the imagined class.

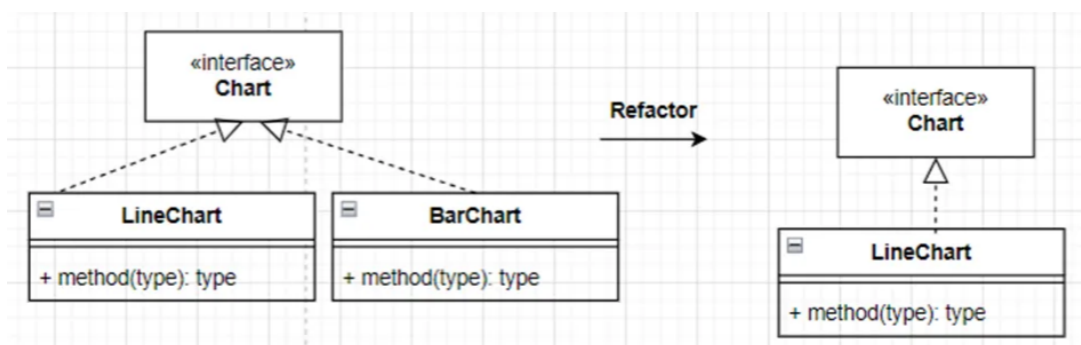


Figure 2: Example of Speculative Hierarchy

1.6 Deep Hierarchy

This smell arises when an inheritance hierarchy is “excessively” deep.

One of the key techniques for effective design of inheritance hierarchies is “apply meaningful generalization.” Instead, when we apply generalization excessively and create a large number of unnecessary intermediate supertypes, it results in a Deep Hierarchy smell. A hierarchy that is excessively deep significantly increases the difficulty in predicting the behavior of code in a leaf type since the type inherits a relatively large number of methods from its supertypes. Consider the following examples that illustrate this problem:

- A method from a supertype close to the root may be overridden many times in the hierarchy; hence, in the leaf types, the semantics of the inherited method may not be clear.

- A field defined in a supertype may be hidden in one of the methods in the subtypes (e.g., a local variable name is provided in a derived method, which happens to be same as a field name inherited from the supertype).

Both these cases can confuse a developer which in turn may lead to the introduction of defects in the software. In other words, a hierarchy that is deep makes the design more complex which leads to poor understandability and a greater possibility of design or implementation errors.

1.6.1 Potential Causes

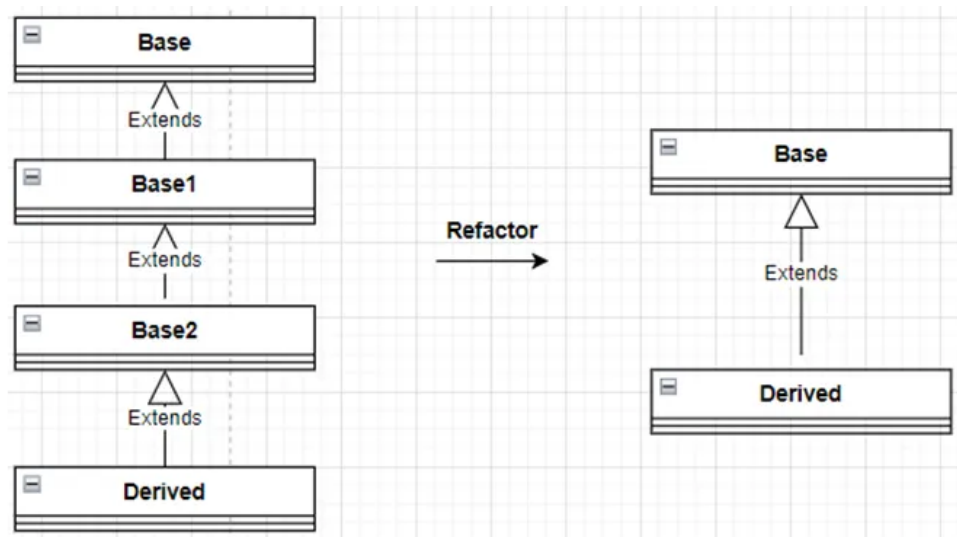
- **Excessive focus on reuse**

Sometimes, developers may indulge in needless generalization to create intermediate types that can be reused. Such excessive focus on reuse at the cost of understandability and usability can lead to the occurrence of a Deep Hierarchy smell.

- **Speculative generalization**

When designers speculatively add supertypes in a hierarchy for imagined future needs, the hierarchy tends to become deep.

1.6.2 Example



1.7 Rebellious Hierarchy

This smell arises when a subtype rejects the methods provided by its supertype(s). In this smell, a supertype and its subtypes conceptually share an IS-A relationship, but some methods defined in subtypes violate this relationship. For example, for a method defined by a supertype, its overridden method in the subtype could:

- throw an exception rejecting any calls to the method
- provide an empty (or NOP i.e., NO Operation) method
- provide a method definition that just prints “should not implement” message
- return an error value to the caller indicating that the method is unsupported

A subtype can mutate behavior it inherits from its supertype. The mutation is harmless when the behavior is enhanced (i.e., specialized) while overriding the supertype methods. However, when the overriding method restricts, or cancels the behavior of the supertype method, the mutation is harmful.

1.7.1 Potential Causes

- **Creating a hierarchy rooted in an existing concrete class**

In real-world projects, as the design evolves, existing concrete classes often become the

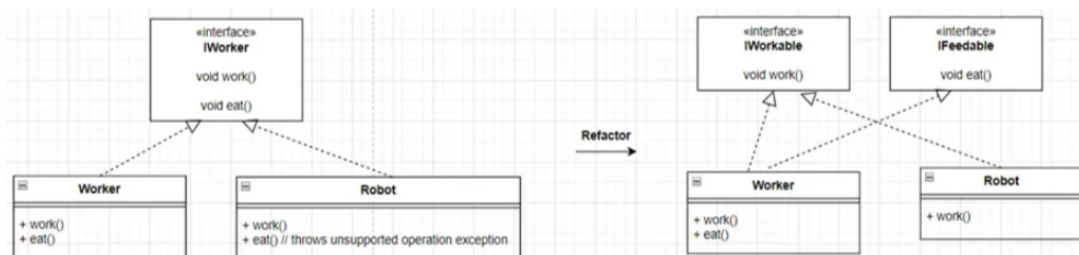
root for new hierarchies. In this context, it is important to perform periodic refactoring of the hierarchy to ensure that the types toward the root are generalized abstractions. In the absence of such refactoring, subtypes may be forced to reject irrelevant methods they inherit from their supertypes.

- **Creating “swiss-knife” types**

When designers try to create a supertype that provides everything, specialized subtypes may be forced to reject some of the methods they inherit from the supertype.

1.7.2 Example

The IWorker interface has work() and eat() method. Worker class can implement the interface and override both methods. But Robot cannot eat() so there it is throwing unsupported operation exception. To refactor it, we should use interface segregation principle, so IWorker is split into IWorkable and IFeedable interfaces. Now Robot class need to implement just IFeedable.



1.8 Broken Hierarchy

This smell arises when a supertype and its subtype conceptually do not share an “IS-A” relationship resulting in broken substitutability. There are three forms of this smell:

- Form 1: The methods in the supertype are still applicable or relevant in the subtype (though the supertype and subtype do not share an IS-A relationship).
- Form 2: The interface (i.e., public methods) of the subtypes includes (by inheritance) the supertype methods that are not relevant or acceptable for the subtypes (note that there is no IS-A relationship); however, the subtypes do not reject those irrelevant or unacceptable methods that are inherited from the supertype.
- Form 3: The subtype implementation explicitly rejects irrelevant or unacceptable methods that are inherited from the supertype.

1.8.1 Potential Causes

- **Inheritance for implementation reuse**

The most common cause of this smell is the use of inheritance for implementation reuse i.e., the subtype inherits from the supertype for reusing the supertype's functionality without sharing a true IS-A relationship with the supertype.

- **Incorrect application of inheritance**

Sometimes, an inexperienced designer may mistakenly invert the inheritance relationship between types.

1.8.2 Example

Square is not a Rectangle. Yes, we can make Square from Rectangle by keeping width and height same value but logically this relation is not correct because it is violation of Liskov Substitution Principle. To refactor it, we can create a common interface i.e., Shape and allow Rectangle and Square to implement it.

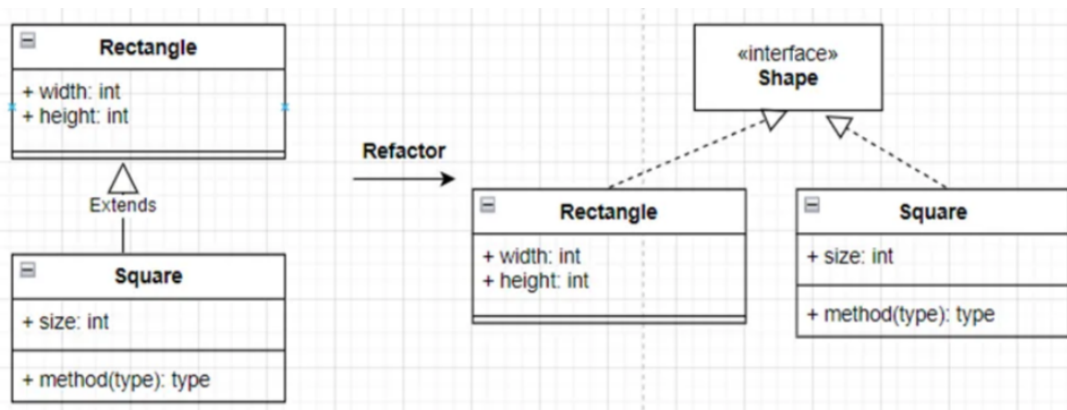


Figure 3: Broken Hierarchy smells

1.9 Multipath Hierarchy

This smell arises when a subtype inherits both directly as well as indirectly from a supertype leading to unnecessary inheritance paths in the hierarchy. When there are unnecessary paths in an inheritance hierarchy (i.e., there is a violation of the enabling technique “avoid redundant paths”), it clutters the hierarchy making it difficult to understand the relationship between the types. Such unnecessary paths can sometimes lead to runtime problems as well.

1.9.1 Potential Causes

Overlooking existing inheritance paths

The most common cause of this smell is when designers often overlook existing inheritance paths while defining a type and unnecessarily inherit a supertype again. The possibility of this mistake is accentuated in inheritance hierarchies that are deep.

1.9.2 Example

